Flash Attention, Slowly

Starting from TinyGPT

SC26 nixnan tutorial

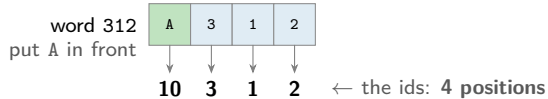
`flash-tutorial`

October 2026

Step 1: a word becomes a list of token ids



A (id 10) = start of a word
F (id 11) = end of a word



the same ids as one-hot rows (4×12)

	0	1	2	3	4	5	6	7	8	9	A	F
A	0	0	0	0	0	0	0	0	0	0	**1**	0
3	0	0	0	**1**	0	0	0	0	0	0	0	0
1	0	**1**	0	0	0	0	0	0	0	0	0	0
2	0	0	**1**	0	0	0	0	0	0	0	0	0

each row has a single 1, in the column of its id

- An id is a **label**: 3 is "token number 3", just as `A` is token number 10. The network never adds or compares ids as numbers.
- The next step turns each id into a vector of 128 numbers by picking one row of a table.

Step 2: each id picks one row of 128 numbers from the embedding table

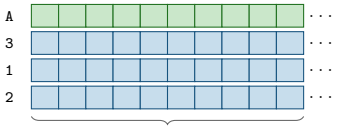
embedding table in `tinygpt.pt` (12×128, after training)

	col 0	col 1	col 2	col 3	⋯	col 127
0	−0.015	+0.010	−0.044	+0.061	⋯	+0.002
1	+0.037	−0.001	−0.059	−0.017	⋯	−0.139
2	+0.013	+0.025	+0.058	−0.007	⋯	+0.004
3	+0.033	+0.016	−0.141	+0.070	⋯	−0.077
4	−0.008	+0.048	+0.008	+0.039	⋯	−0.031
5	+0.030	+0.016	+0.027	+0.009	⋯	−0.013
6	+0.008	+0.066	+0.063	−0.054	⋯	+0.012
7	+0.075	+0.091	+0.013	−0.058	⋯	+0.014
8	+0.035	−0.010	+0.030	−0.076	⋯	+0.022
9	−0.014	−0.014	+0.020	+0.049	⋯	+0.078
A	−0.001	+0.000	−0.006	−0.012	⋯	−0.014
F	+0.010	+0.016	−0.020	+0.020	⋯	+0.005

12 rows: one per token

copy rows → A, 3, 1, 2

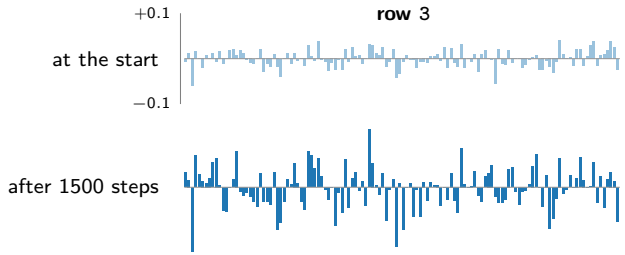
what block 0 gets for `A312`



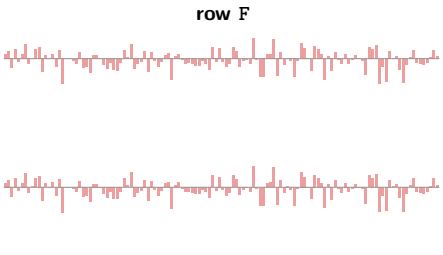
one-hot (4×12) × table (12×128) = these 4 rows (4×128): a matrix multiply that just copies rows

- Every entry is one **FP32 number** (32 bits). A row is 128 numbers, i.e. $128 \times 32 = 4096$ bits, not 128 bits.
- Row 3 is what the network means by 3. Its 128 numbers are coordinates of a point; no single column means anything by itself.

Where the numbers come from: random at first, then moved by training



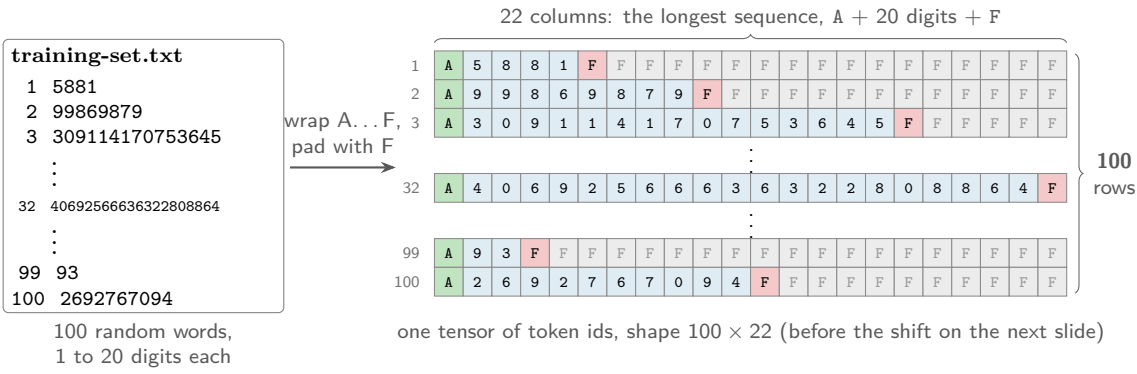
spread out: values now −0.14 to +0.13



unchanged: bit-for-bit the random start

- **Start:** every entry is drawn at random from a normal distribution with mean 0 and spread 0.02 (seed 0).
- **Training:** each of the 1500 AdamW steps nudges every entry in the direction that lowers the loss, exactly like TinyGPT's other parameters (the table is 1,536 of 791,808).
- **Why `F` never moves:** `F` is only ever an *input* after a word has ended, where the target is ignored (−100). Causal attention lets it affect only those positions, so its gradient is exactly zero.

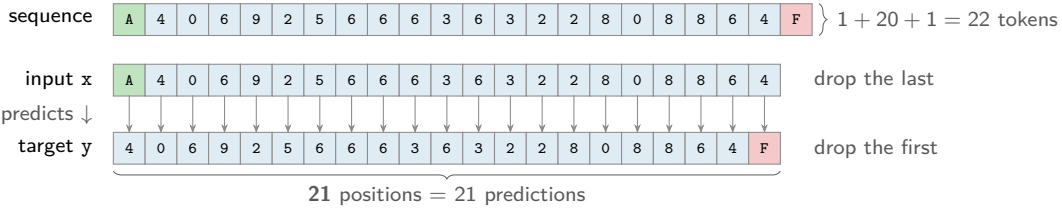
Where the batch size 100 comes from: one row per training word



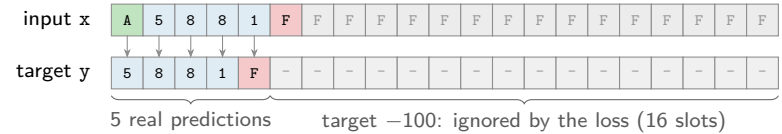
There is no batch-size setting: `train.py` stacks *every* word of the training set into one batch and takes all 1500 optimizer steps on that same batch (no shuffling, no sampling).

Where 21 comes from: each position predicts the next token

word 32 (20 digits, the longest)

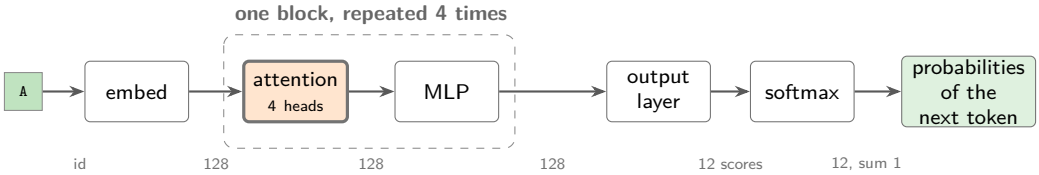


word 1 (5881): padded to the same 21

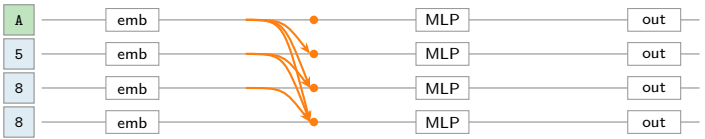


So the training batch is 100×21: 100 words, 21 positions each.

TinyGPT in miniature: what happens to one position



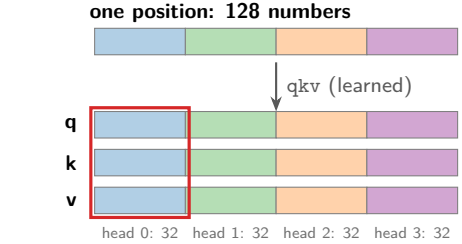
four positions of `A5881F`, side by side:



only the orange attention arrows cross between positions, and only from **earlier** ones

Every box except attention works on each position by itself. Attention is how position 3 learns what came at positions 0, 1, 2.

Inside attention: the 128 numbers are split into 4 heads of 32



head 0 works alone, on its 32-number slices

for every position i (query) and every **earlier or same** position j (key):

$\text{score}_{ij} = q_i \cdot k_j / \sqrt{32}$ (32 products, summed)

weights: softmax of the scores along row i

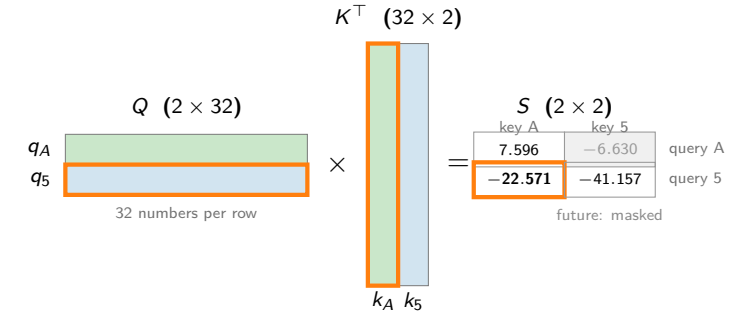
output_i = weighted blend of the v_j (32 numbers)

heads 1, 2, 3: the same, on their own slices, in parallel

glue back to 128, then `proj` mixes the heads → 128 again

A head does *not* make a prediction. It only decides what each position borrows from earlier positions; the prediction comes out of the output layer at the very end. ("Head" and "output head" are unrelated names.)

Attention on A5, step 1: scores $S = Q\,K^\top$ (head 0, block 0)



In training this is the **top-left** 2×2 **corner** of each head's 21×21 grid.

Rows A and 5 may only use keys A and 5 (causal), so the other 19 positions of the word cannot change these numbers.

Each entry: one query row meets one key column.

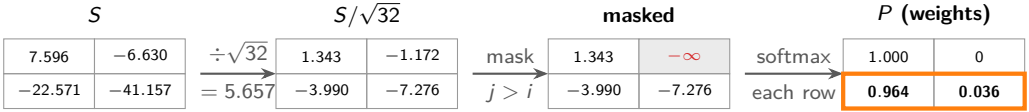
one pair:

	1	2	3	4	···	32
q_5	0.673	3.543	5.020	−2.008	···	2.209
k_A	6.453	−9.391	−0.717	−4.605	···	−0.437
	×	×	×	×		×
products	4.342	−33.271	−3.598	9.247	···	−0.965

sum of 32 → **−22.571** $= S_{5A}$

Head 0 of block 0, trained `tinygpt.pt`; q, k after RoPE, all rounded to FP16 as handed to the flash kernel. $q_5 \cdot k_A = -22.6$ is larger than $q_5 \cdot k_5 = -41.2$: query 5 prefers key A.

Step 2: divide by $\sqrt{32}$, mask the future, softmax each row



Why $\sqrt{32}$? A score is a sum of 32 products. If they were independent with spread 1, the sum's spread would be $\sqrt{32}$ (spreads add in quadrature). Dividing by $\sqrt{32}$ keeps scores near ± 1 at the start of training, so the softmax is not already one-hot and gradients can flow.

The 32 is the head size, not the 128 of the whole position.

Row 5 by hand (row A has one key, so its weight is 1):

subtract the row max: $(-3.990, -7.276) - (-3.990) = (0, -3.286)$

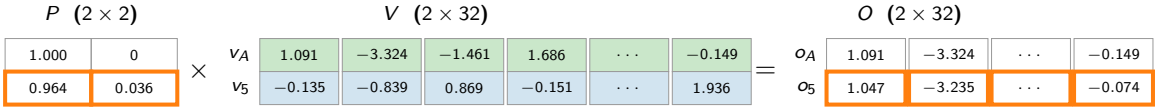
exponentiate: $(1,\ e^{-3.286}) = (1,\ 0.0374)$

divide by the sum 1.0374: $(0.964,\ 0.036)$

These are the 0.96/0.04 of the $n = 2$ snapshot.

Subtracting the row max changes nothing mathematically but keeps $e^x \leq 1$ in FP16/FP32. Flash keeps exactly this max as a *running* max while it streams over key blocks (the online softmax).

Step 3: $O = P\,V$, each output row is a weighted blend of value rows



Row 5, column by column: $o_5 = 0.964\,v_A + 0.036\,v_5$

col 1: $0.964 \times 1.091 + 0.036 \times (-0.135) = 1.047$

col 2: $0.964 \times (-3.324) + 0.036 \times (-0.839) = -3.235$

col 32: $0.964 \times (-0.149) + 0.036 \times 1.936 = -0.074$

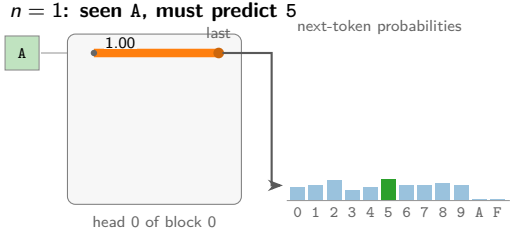
Row A has weight 1 on itself, so $o_A = v_A$ exactly. Position 5 borrows almost everything from A: this head looks back at the start symbol.

Then: heads 1–3 do the same on their own slices; the four 2×32 outputs are glued into 2×128, mixed by `proj`, and added back to x.

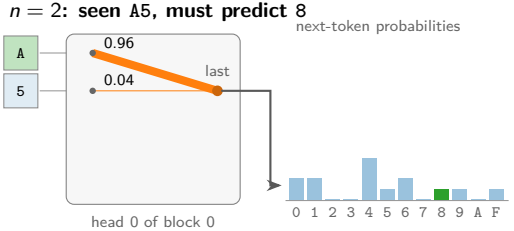
Sizes in training: per word and head, S, P are 21×21 and O is 21×32; per step 100×4 of each, in each of 4 blocks.

Flash attention does steps 1–3 tile by tile on tensor cores (FP16 in, FP32 accumulate): a tile of S lives only in registers, is scaled, masked and exponentiated, multiplied by a tile of V and added into O. S and P are never written to memory.

Training snapshots on A5881F: $n = 1$ and $n = 2$ tokens seen



$p(5) = 0.14$, loss $-\ln 0.14 = 1.97$
14 of the 100 words start with 5: the model has learned how often each first digit occurs

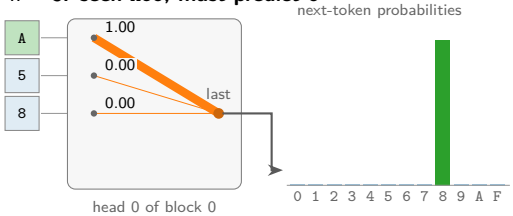


$p(8) = 0.07 = 1/14$, loss 2.64
of the 14 words starting with 5, only one goes on with 8 (and four with 4): A5 alone can't tell

Orange lines: how much the last position's head 0 (block 0) takes from each position seen so far (the numbers sum to 1). A position can never look at a later one. All values are from the trained `tinygpt.pt`.

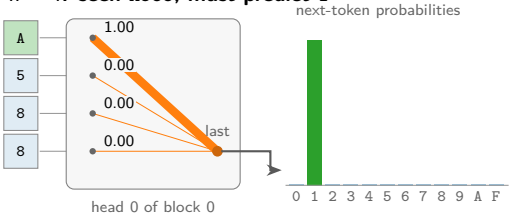
Training snapshots on A5881F: $n = 3$ and $n = 4$ tokens seen

$n = 3$: **seen A58, must predict 8**



$p(8) = 1.00$, loss 0.00
5881 is the only word starting 58,
so three tokens pin it down

$n = 4$: **seen A588, must predict 1**



$p(1) = 1.00$, loss 0.00
(and at $n = 5$, A5881 $\rightarrow$ F, also with $p = 1.00$)

Block 0's head 0 mostly looks back at A; other heads and later blocks pick up the digits. The loss pushes p(correct next token) up at every position, and the gradient flows back through all of it, the embedding rows included.

Training does every snapshot at once

head 0, block 0, word `A5881F`: one row per snapshot

	A	5	8	8	1	F	
A	1.00						→ predict 5
5	0.96	0.04					→ predict 8
8	1.00	0.00	0.00				→ predict 8
8	1.00	0.00	0.00	0.00			→ predict 1
1	0.01	0.88	0.00	0.00	0.11		→ predict F
F	0.40	0.06	0.08	0.03	0.09	0.33	(ignored)

row n = snapshot n:
the last position's weights;
gray = later positions,
never looked at

One forward pass computes all the snapshots together:

- every word: **21** positions = 21 snapshots (one row each)
- every head: one 21×21 grid per word, like the one at left
- per step: 100 words $\times$ 4 heads $\times$ 4 blocks = **1,600 grids**
- loss = average of $-\ln p$(correct next token) over all real positions
- one backward pass, one AdamW step; repeat 1500 times

Generation is different: batch 1, one snapshot at a time. 312 → `A312` (4 positions) in one call, then one new position per call, reusing the stored keys and values of earlier positions.